

AI Engineer Coding Exercise

Overview

This exercise is designed to evaluate your skills in developing AI applications that use Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG). You'll demonstrate your ability to integrate key components such as:

- **LLMs:** using pre-trained models to generate answers.
- **Prompt engineering:** creating clear and effective instructions to guide the model.
- **Embeddings and vector databases:** transforming text into vectors and using a vector database for efficient information retrieval.

Core task

Your main task is to create a service that exposes an HTTP endpoint that receives a user's question and returns an answer generated by an LLM, using a provided knowledge base as context.

- You can use any programming language (Python, Node.js, etc.) and libraries you prefer.
- Use the following knowledge base as the source of information for the LLM:

"BEON.tech mission is to place the brightest tech talent in the most disruptive and innovative U.S. companies."

"BEON.tech offer IT staff augmentation services for every modern tech need, from backend and frontend to AI, machine learning, DevOps and QA."

"At BEON.tech, building software means far more than just filling roles – it's about creating strong relationships, empowering careers and helping people grow."

- You might consider using an LLM that is free, lightweight and open-source, with no API key required for use. The model should ideally be runnable locally or via an accessible service. For example, you might consider models available on Hugging Face, but feel free to use any approach you like.
- Apply prompt engineering techniques to ensure the model's response is concise, relevant and that it relies on the provided context.
- The endpoint should receive a question in the request and return the answer in a structured JSON format.

Bonus task

For this part, upgrade your design to a full RAG system. Instead of sending the entire knowledge base to the LLM for every request, perform the following steps:

- Create embeddings for each snippet in the knowledge base.
- Store these embeddings in a local vector database (e.g., ChromaDB or any other of your choice).
- When the service receives a question, generate an embedding for that question.
- Use the vector database to retrieve the most relevant text snippets from the knowledge base based on the user's question.
- Pass only the relevant snippets to the LLM along with the question.

Evaluation criteria

Your solution will be evaluated based on the following:

- **Functionality:** does the service correctly answer questions using the knowledge base?
- **Design:** is the architecture clear and extensible?
- **Technology usage:** do you demonstrate a practical understanding of LLMs, RAGs, embeddings and vector databases?
- **Prompt engineering:** is the prompt effective and does it prevent the model from "hallucinating" or inventing information?
- **Code quality:** is the code clean, well-structured and easy to read?